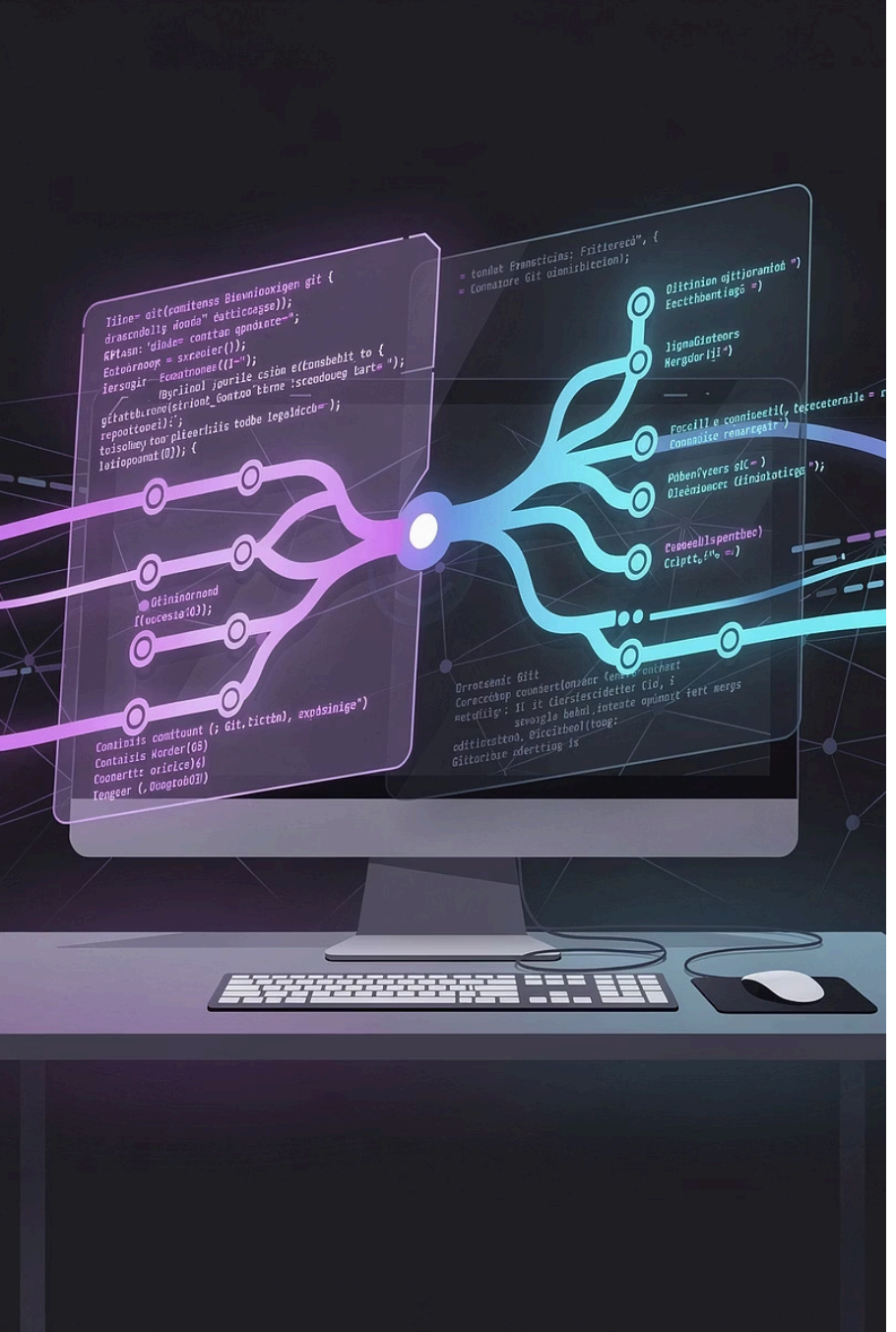


שיעור 1

Git & GitHub

כל מה שצריך לדעת כדי לנהל קוד, לשתף פעולה בצוות ולעבוד כמו בעולם האמיתי

אז מה זה Git?

היא מערכת **ניהול גרסאות מבוזרת** בקוד פתוח, שנוצרה ב-2005 על ידי לינוס טורוולדס, יוצר לינוקס. Git מטרתה לסייע למפתחים בניהול קוד מקור, תיאום עבודה צוותית ומעקב אחר כל שינוי בקבצי התוכנה.

בניגוד למערכות מרכזיות, ב-Git **כל ספרייה בכל מחשב** נחשבת למאגר נתונים מלא, הכולל תיעוד היסטוריה שלם ואפשרויות מעקב — ללא תלות ברשת או בשרת מרכזי. המשמעות? אתם יכולים לעבוד גם אופליין ולסנכרן מאוחר יותר.

מעקב שינויים

כל שינוי מתועד עם חותמת זמן והודעה

עבודת צוות

מאפשר למפתחים לעבוד במקביל על אותו פרויקט

גיבוי מבוזר

כל עותק מקומי הוא גיבוי מלא של הפרויקט



מה זה GitHub?

היא **פלטפורמה מבוססת אינטרנט** רבת עוצמה, המאפשרת למפתחים לנהל, לאחסן ולשתף GitHub עם ממשק ויזואלי נוח, כלים לסקירת קוד, מעקב Git פעולה בפרויקטי תוכנה. היא משלבת את כוחו של .אחר באגים ובעיות, ומערכת אוטומציה מתקדמת

מעבר לאחסון קוד, GitHub מספקת **מרחב קהילתי** לשיתוף קוד פתוח — מיליוני מפתחים ברחבי העולם משתמשים בה מדי יום כדי לתרום לפרויקטים, ללמוד מקוד של אחרים ולבנות יחד תוכנה טובה יותר.

בקרת גרסאות

מעקב מלא אחר כל שינוי בקוד עם היסטוריה מפורטת

מעקב בעיות

מערכת Issues מובנית לניהול באגים ומשימות

סקירת קוד

כלי Pull Request לבדיקה ואישור שינויים לפני מיזוג

קוד פתוח

קהילה עולמית של מיליוני מפתחים שמשתפים ותורמים

מה בין Git ל-GitHub?

שאלה נפוצה בקרב מפתחים מתחילים. בואו נבהיר אחת ולתמיד — **Git הוא הכלי** שרץ על המחשב שלכם, ו-**GitHub הוא השירות** שמאחסן את הקוד בענן ומאפשר שיתוף פעולה.

1

Git

- תוכנה שמותקנת מקומית על המחשב
- מערכת ניהול גרסאות מבוזרת
- עובדת גם ללא חיבור לאינטרנט
- שורת פקודה (CLI)

2

GitHub

- פלטפורמה אינטרנטית מבוססת ענן
- ממשק ויזואלי לניהול Git
- דורשת חיבור לאינטרנט
- כלים לשיתוף פעולה וקהילה

💡 **בקצרה:** Git = מנוע | GitHub = לוח המחוונים. אפשר להשתמש ב-Git בלי GitHub, אבל GitHub בלי Git? לא באמת.



מושגים בסיסיים ב-Git

שלוש התחנות של Git

חשבו על Git כבעלת **שלוש תחנות עיקריות** שכל שינוי עובר דרכן לפני שהוא נשמר לתמיד:

01

Working Copy

הקבצים בתיקיית הפרויקט שלנו

02

Staging Copy

קבצים שהוכנו למעקב של Git

03

Committed Copy

שינויים שנשמרו סופית ב-repo

מהו Repository?

הוא מבנה נתונים המאחסן (**repo** או בקיצור) Repository מטא-דאטה על קבוצת קבצים ותיקיות, בצורה של עץ תיקיות.

במילים פשוטות — **שם נמצא ומתבצע כל ניהול הגרסאות** של הקוד שלנו. כל פרויקט שאנחנו מנהלים עם Git חי בתוך repo.

שלוש התחנות — לעומק

כל שינוי שנבצע בקוד עובר מסע מ-Working Copy דרך Staging ועד Commit. הבנת התהליך הזה היא **המפתח לשליטה ב-Git**.



Committed Copy

לאחר `git commit`, השינוי נשמר ב-repo. נוצר "תמונת מצב" של כל הקבצים עם הודעה קצרה שמתארת מה השתנה — כמו נקודת שחזור.

Staging Copy

לאחר שינוי, אנו "מוסיפים" את הקבצים למעקב באמצעות `git add`. הקבצים עוברים ל-Staging — אזור הכנה. **שימו לב:** עדיין לא ביצענו Commit!

Working Copy

הקבצים כפי שהם על המחשב שלכם. כאשר אתם עורכים ושומרים קובץ — ה-Working Copy השתנה. Git מזהה שינוי אך עדיין לא עוקב אחריו.

פקודות בסיסיות ב-Git

הגיע הזמן ללכלך את הידיים! נתחיל מהפקודה הראשונה — **אתחול repository מקומי**.

git init

הפקודה `git init` מאתחלת repository מקומי חדש בתיקייה שבה אנו נמצאים. לדוגמה, בתיקייה `myRepo` נפתח את שורת הפקודה ונכתוב:

```
cd myRepo
git init
```

מיד נקבל הודעה שנוצר repository חדש בתיקייה. 🎉

> 💡 **טיפ מעשי:** ברוב המקרים נעדיף ליצור את ה-repo ב-GitHub קודם, ואז לשכפל אותו למחשב באמצעות `git clone` — במקום לאתחל מקומית ולדחוף אחר כך. כך נחסוך שלבי הגדרה מיותרים.

git add ו-git commit

שתי הפקודות הכי חשובות בזרימת העבודה היומיומית — **הכנת שינויים** ו**שמירתם** ב-repo.

1

git add — הכנת שינויים

מעבירה שינויים מתיקיית העבודה אל ה-Staging Area, כדי שייכללו בקומיט הבא:

```
git add example.txt   # מוסיף קובץ מסוים
git add .             # מוסיף את כל השינויים
```

2

git commit — שמירת שינויים

שומרת את מה שנמצא ב-Staging אל ה-repo המקומי ויוצרת קומיט חדש:

```
git commit -m "change example file"
```

3

git status — מה המצב?

מראה לנו בדיוק מה קורה ב-repo: אילו קבצים שונו, מה ב-Staging ומה עדיין לא:

```
git status
```

דוגמה מעשית — זרימת עבודה מקומית

בואו נראה את כל הפקודות בפעולה. נניח ששינינו את הקובץ `example.txt` בתיקיית `myRepo`:

שלב 1: עריכת קובץ

שינינו את `example.txt` ושמרנו. ה-Working Copy השתנה.

1

שלב 2: בדיקת סטטוס

מריצים `git status` ורואים שהקובץ מופיע באדום — טרם נוסף ל-Staging.

2

שלב 3: הוספה ל-Staging

מריצים `git add example.txt` והקובץ עובר לירוק.

3

שלב 4: ביצוע Commit

מריצים `git commit -m "change example"` — מצוין! הגרסה המעודכנת נשמרה ב-repo המקומי. ✅

4

```
git status
git add example.txt
git commit -m "change example file content"
```

GitHub — יצירת Repo

לאחר שהבנו כיצד Git עובד מקומית, הגיע הזמן לחבר את העבודה שלנו ל-**שרת מרוחק** ב-GitHub. נלמד כיצד ליצור repository חדש ולשכפל אותו למחשב.

1 יצירת Repo ב-GitHub

נכנסים ל-GitHub, לוחצים על **New Repository**, בוחרים שם, הגדרות ראשוניות (ציבורי/פרטי, קובץ README) ויוצרים.

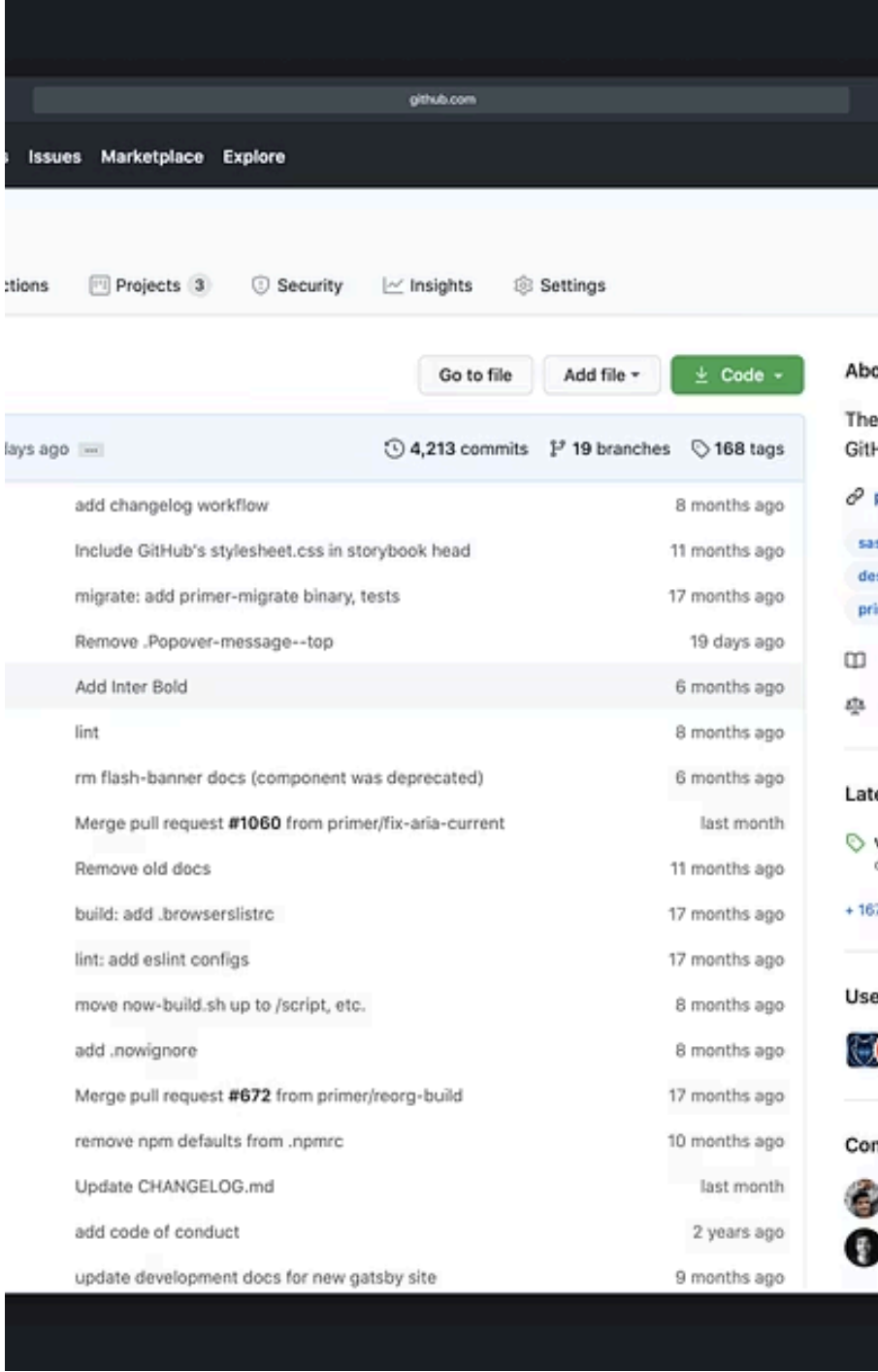
2 הורדה למחשב — git clone

מעתיקים את כתובת ה-repo ומריצים: `git clone https://github.com/user/myRepo.git`. הפקודה יוצרת תיקייה חדשה בנתיב שאנו נמצאים בו.

3 כניסה לתיקייה

כדי לעבוד על הפרויקט ולבצע פקודות Git, יש להיכנס לתיקייה: `cd myRepo`. רק מתוך התיקייה Git ידע לזהות את ה-repo.

⚠️ **שימו לב!** הפקודה `clone` יוצרת תיקייה חדשה בנתיב שבו אתם נמצאים. וודאו שאתם במיקום הנכון לפני ההרצה.



git push ו-git pull

כעת שה-repo המקומי מחובר ל-GitHub, נלמד כיצד **לסנכרן שינויים** בין המחשב שלנו לשרת המרוחק. שתי הפקודות האלה הן הלב של עבודת צוות.

git pull — משיכת שינויים

מה קורה אם מישהו אחר בצוות עדכן את הקוד? הפקודה `git pull` מושכת את השינויים מהשרת המרוחק אל המחשב שלנו, ומסנכרנת את העותק המקומי עם הגרסה העדכנית ביותר.

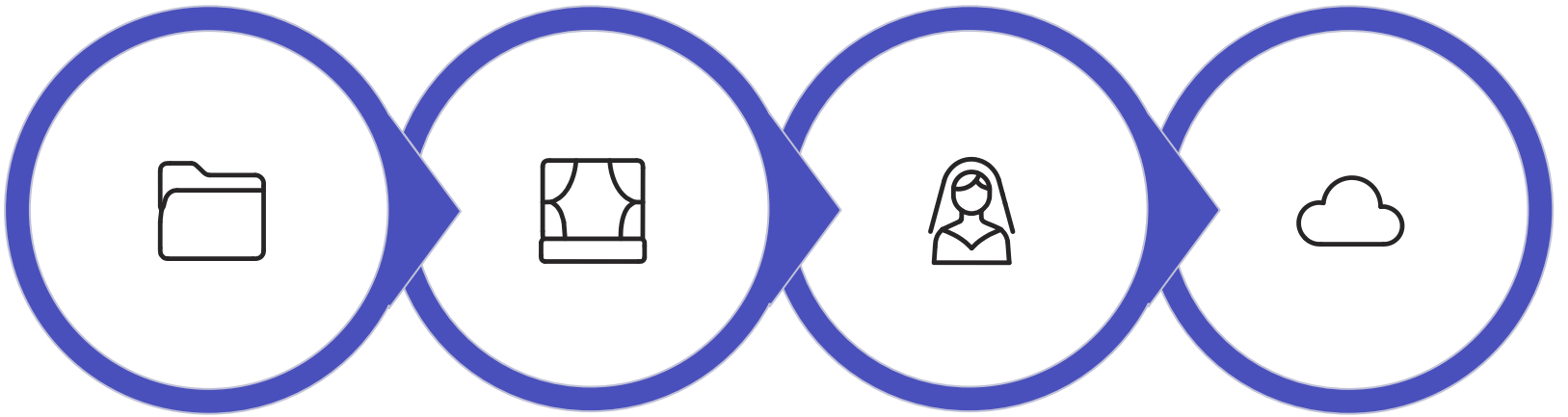
git push — דחיפת שינויים

לאחר שביצענו `commit` מקומי, הפקודה `git push` דוחפת את השינויים אל ה-repo המרוחק ב-GitHub. כך חברי הצוות יכולים לראות ולגשת לעדכונים שלנו.

```
git push origin main # דוחף שינויים ל-branch הראשי
git pull origin main # מושך שינויים מ-branch הראשי
```

התהליך המלא — מהשינוי ועד ל-Repo

בואו נסכם את כל מה שלמדנו בתרשים אחד שמראה את **המסע של שינוי בקוד** — מרגע העריכה במחשב שלכם ועד שהוא מגיע ל-repo המרוחק ב-GitHub.



כל שינוי עובר בדיוק את אותו המסלול: **עריכה → הוספה → קומיט → דחיפה**. ברגע שהתהליך הזה נהפך לטבע שני, אתם שולטים ב-Git.

צוות מנצח ✌️

כמו שאמרנו, GitHub נועד בראש ובראשונה ל**עבודת צוות ושיתוף קוד**. הוספת חברי צוות ל-repo היא פשוטה ומהירה.

כניסה להגדרות

נכנסים ל-repo ב-GitHub ולוחצים על **Settings** בתפריט העליון

הוספת שותפים

עוברים ל-**Collaborators** ולוחצים על **Add People**. מכניסים את שם המשתמש של השותף.

אישור הזמנה

המוזמן מקבל הזמנה במייל ובממשק GitHub. לאחר אישור — הוא יכול לבצע שינויים, לדחוף קוד ולשתף פעולה באופן מלא. 🎉

🔒 **טיפ אבטחה:** ניתן לקבוע הרשאות שונות לכל Collaborator — קריאה בלבד, כתיבה, או הרשאות מנהל מלאות. מומלץ לתת את ההרשאה המינימלית הנדרשת.





gitignore. — נבין אחת ולתמיד!

ברוב הפרויקטים יש קבצים ותיקיות ש**לא נרצה** שיעלו ל-repo — כמו תיקיות התקנה, קבצי סביבה רגישים, או קבצים זמניים. הפתרון? קובץ `.gitignore`.

ומה עם ספריות שלא העלינו?

אם התקנו ספריות (כמו דרך npm) והן ב-`.gitignore`, כיצד מפתח אחר ידע מה להתקין?

לשם כך קיים הקובץ `package.json` שמתעד את כל התלויות. כשעוברים למחשב חדש, פשוט מריצים:

```
npm install
```

או בקיצור `npm i` — והכול מותקן אוטומטית! ✨

כיצד זה עובד?

יוצרים את הקובץ באמצעות הפקודה:

```
touch .gitignore
```

כל שורה בקובץ מגדירה דפוס של קבצים או תיקיות שגיט יתעלם מהם. לדוגמה:

```
node_modules/
dist/
.env
*.log
```

סיכום — עכשיו אתם מוכנים! 🚀

עברנו יחד מסע מקיף בעולם Git ו-GitHub. בואו נחזור על **הנקודות המרכזיות**:

1

ניהול גרסאות = Git

מערכת מבוזרת ומקומית לתיעוד כל שינוי בקוד

2

שיתוף = GitHub פעולה

פלטפורמה בענן לאחסון, שיתוף וניהול פרויקטים

3

שלוש תחנות

Working → Staging → Committed — כל שינוי עובר מסלול קבוע

4

פקודות מפתח

ארגז הכלים שלכם — init, add, commit, push, pull, clone, status

💜 **ש"ב:** https://github.com/Chavi-GIT/Lesson1

